

Rejection Sampling

Gaussian Envelope vs. Piecewise Log-Affine

Overview

- ▶ The target distribution
- ▶ Method 1: Gaussian envelope
- ▶ Method 2: Piecewise log-affine envelope
- ▶ Comparison

The Target Distribution

We want to sample from

$$f(y) \propto q(y) = \exp \left(\sum_{i=1}^m (yz_i x_i - e^{yx_i}) \right) \mathbf{1}_{\{y \geq 0\}}, \quad m = 100$$

► Data: 100 observations (z_i, x_i) , with $x_i > 0$

Goal: sample from f via rejection sampling, using two different envelope constructions

- A Gaussian envelope
- A piecewise log-affine envelope

Why a Gaussian Envelope?

We look for a Gaussian density p that dominates q up to a constant, $\alpha' q(y) \leq p(y) \Leftrightarrow \log(q(y)) \leq \log(p(y)) - \log(\alpha')$.

We have that the log of the Gaussian density is

$$\log p(y) = -\frac{1}{2} \log(2\pi\sigma^2) - \frac{(y - \mu)^2}{2\sigma^2}.$$

Deriving the Envelope

Since $yx_i \geq 0$, the inequality $e^t \geq 1 + t + t^2/2$ gives

$$\begin{aligned}\log q(y) &= \sum_{i=1}^{100} (yz_i x_i - e^{yx_i}) \leq -100 + Ay - \frac{B}{2}y^2 \\ &= -100 + \frac{A^2}{2B} - \frac{B}{2} \left(y - \frac{A}{B}\right)^2\end{aligned}$$

where $A = \sum_{i=1}^{100} x_i(z_i - 1)$ and $B = \sum_{i=1}^{100} x_i^2$.

Matching to $-\frac{(y - \mu)^2}{2\sigma^2}$ gives $\mu = A/B$, $\sigma^2 = 1/B$. Our $p(y)$ is the **normalized** Gaussian density, so:

$$\alpha' = \exp \left(100 - \frac{A^2}{2B} \right) / (\sigma \sqrt{2\pi})$$

First Implementation: Gaussian Envelope

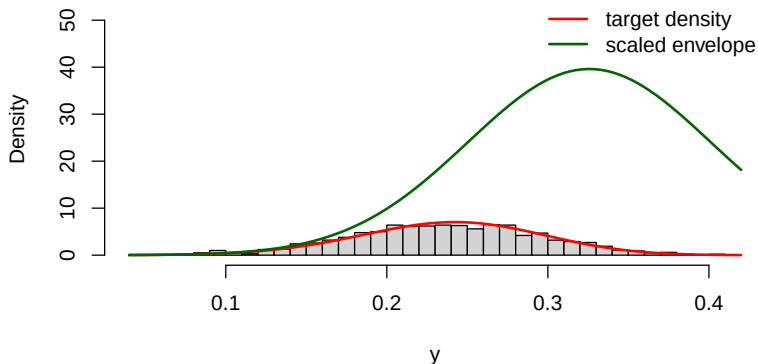
```
#A <- sum(dat$x * (dat$z - 1))      ,      B <- sum(dat$x^2),
# mu <- A / B      ,      sigma <- sqrt(1 / B)
log_alpha_prime <- 100 - A^2 / (2 * B) - log(sigma * sqrt(2 * pi))

log_f_vec <- function(y) {
  sapply(y, function(yi) sum(dat$z * dat$x * yi - exp(dat$x * yi)))
}
log_accept_gaussian <- function(y) {
  log_f_vec(y) + log_alpha_prime -
    dnorm(y, mean = mu, sd = sigma, log = TRUE)
}
gaussian_rproposal <- function(n) {
  rnorm(n, mean = mu, sd = sigma)
}
log_rejection_sample_random <- function(N, rproposal, log_accept) {
  y <- rproposal(N)
  u <- runif(N)
  accept <- log(u) <= log_accept(y)
  y[accept]
}
log_rejection_sample_vec <- rng_vec(log_rejection_sample_random)
```

$$U \leq \alpha' f(y)/p(y) \Leftrightarrow \log(U) \leq \log f(y) + \log \alpha' - \log p(y)$$

Correctness Check: $N = 1000$ Samples

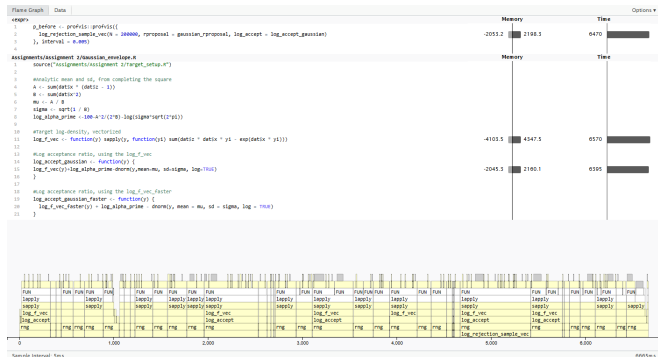
Samples vs target vs envelope



Empirical acceptance rate (fraction accepted out of 200,000 raw proposals): 0.1341

Profiling: Where Does the Time Go?

Profiling `log_rejection_sample_vec` (200,000 samples) with `profvis`/`Rprof` — the call is shown at the top of the screenshot:



6570 of 6665 ms of self time 98.5% is on one line not the math itself, but calling a fresh R closure once per proposal via `apply`.

Vectorizing log_f

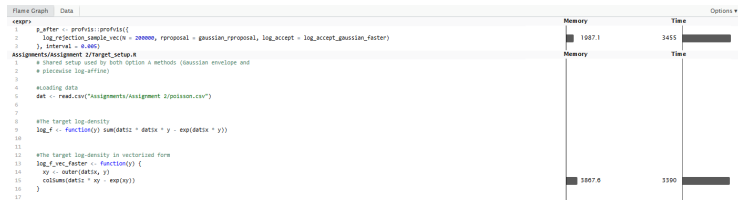
With a 13.4% acceptance rate, roughly 1.5 million proposals get evaluated to produce 200,000 accepted samples, each one re-running `sample` and re-fetching `dat$z/dat$x` from scratch.

Idea: replace the one-proposal-at-a-time `sample` with a single matrix computation covering *all* proposals in a batch at once.

```
log_f_vec_faster <- function(y) {  
  xy <- outer(dat$x, y)  
  colSums(dat$z * xy - exp(xy))  
}
```

► Exact same output as the original (verified: zero difference)

Profiling log_f_vec_faster: the `sample` bottleneck is gone — runtime is now genuinely inside the vectorized math itself:



Two C++ Implementations

General (callback into R for every proposal):

```
NumericVector rejection_sample_cpp(  
    int N, Function rproposal, Function log_accept) {  
    NumericVector out(N);  
    double y;  
    bool reject;  
  
    for (int i = 0; i < N; ++i) {  
        do {  
            y = as<double>(rproposal(1));  
            reject = log(R::runif(0, 1)) >  
                as<double>(log_accept(y));  
        } while (reject);  
        out[i] = y;  
    }  
  
    return out;  
}
```

Specialized (target/envelope hardcoded, zero R calls):

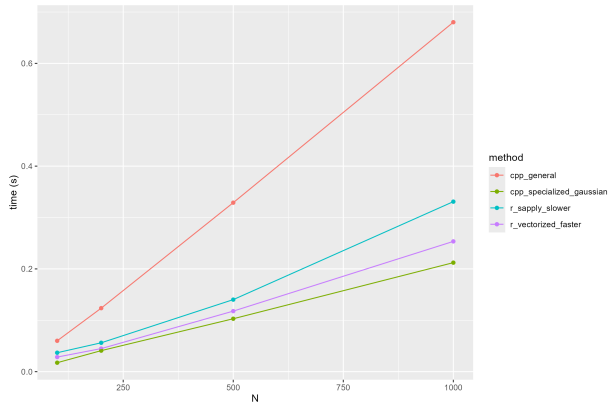
```
NumericVector gaussian_rejection_cpp(  
    int N, NumericVector z,  
    NumericVector x, double mu,  
    double sigma, double log_alpha_prime) {  
    int n = z.size();  
    NumericVector out(N);  
    double y, log_fy, log_gy;  
    bool reject;  
  
    for (int i = 0; i < N; ++i) {  
        do {  
            y = R::rnorm(mu, sigma);  
            log_fy = 0.0;  
            for (int k = 0; k < n; ++k)  
                log_fy += z[k]*x[k]*y - exp(x[k]*y);  
            log_gy = R::dnorm(y, mu, sigma, true);  
            reject = log(R::runif(0, 1)) >  
                (log_fy + log_alpha_prime - log_gy);  
        } while (reject);  
        out[i] = y;  
    }  
  
    return out;  
}
```

Benchmarking All Four Versions



The CPP: Generality costs speed, since the general version pays R-callback overhead on every proposal.

Benchmarking Across N



- ▶ `cpp_specialized_gaussian` and `r_vectorized_faster` stay close together
- ▶ `cpp_general` scales the worst, `r_sapply_slower` (original) is a clear second-worst

Piecewise Log-Affine Envelope

Following Section 6.2.1: write $\ell(y) = \log q(y)$. Then

$$\ell'(y) = \sum_i x_i (z_i - e^{yx_i}), \quad \ell''(y) = -\sum_i x_i^2 e^{yx_i} < 0.$$

So ℓ is **concave** — hence we can use it as our proposal for the piecewise log-affine envelope.

The tangent at t_j has slope $a_j = \ell'(t_j)$ and intercept $b_j = \ell(t_j) - a_j t_j$.

Building the Envelope

Adjacent tangents intersect at

$$c_j = \frac{b_{j+1} - b_j}{a_j - a_{j+1}}.$$

With $c_0 = 0$ and $c_k = \infty$, these define the pieces. The lowest tangent gives $V(y) = a_j y + b_j$ on $[c_{j-1}, c_j]$, hence

$$\alpha' q(y) \leq e^{V(y)}.$$

Tangency means equality at the chosen points $\Rightarrow$ we can use $\alpha' = 1$ as this works already.

Sampling from the Envelope

For a piece with endpoints L, R , its area is

$$R_j = e^{b_j} \frac{e^{a_j R} - e^{a_j L}}{a_j} \quad (\text{or } e^{b_j}(R - L) \text{ if } a_j = 0).$$

- ▶ These areas give the piecewise-exponential CDF, inverted per-piece to sample
- ▶ The last slope must be negative for the unbounded right tail to be integrable

How Many Intervals?

The book gives no formula — only a qualitative trade-off:

“By increasing the number of z 's we can make this envelope as tight as we want. This will lower the rejection probability but also increase the complexity of simulating from the proposal.”

Our own measurements confirm this is genuinely an empirical question, not a closed-form one — very few points is clearly bad, but beyond a modest number, more points barely help while adding overhead.

Implementing the Log-Affine Envelope (1/2)

Building the piecewise tangent-line and evaluating it at a point:

```
build_envelope <- function(y_points) {  
  y_points <- sort(y_points)  
  m <- length(y_points)  
  a <- log_f_prime_vec(y_points)  
  b <- log_f_vec_faster(y_points) - a * y_points  
  z <- (b[-1] - b[-m]) / (a[-m] - a[-1])      # tangent intersections c_j  
  list(a = a, b = b, z = c(0, z, Inf))  
}  
  
eval_V <- function(envelope, y) {  
  i <- findInterval(y, envelope$z)  
  envelope$a[i] * y + envelope$b[i]  
}
```

Since tangency gives equality at each t_j , $\alpha' = 1$ — no separate normalizing-constant step is needed.

Implementing the Log-Affine Envelope (2/2)

Sampling from the piecewise-exponential envelope, then the accept/reject step:

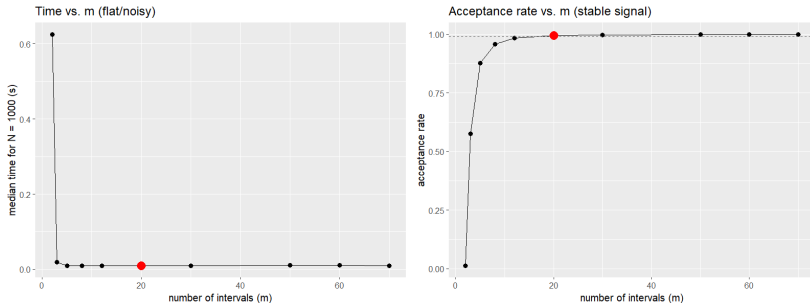
```
sample_from_envelope_vec <- function(envelope, n) {  
  a <- envelope$a; b <- envelope$b; z <- envelope$z  
  m <- length(a)  
  R <- numeric(m) # per-piece area under exp(V)  
  for (i in seq_len(m)) {  
    lo <- z[i]; hi <- z[i + 1]  
    R[i] <- if (is.infinite(hi)) -exp(a[i]*lo + b[i]) / a[i]  
      else (exp(a[i]*hi + b[i]) - exp(a[i]*lo + b[i])) / a[i]  
  }  
  Q <- cumsum(R); d <- Q[m]  
  target_area <- runif(n) * d  
  i <- pmin(findInterval(target_area, Q) + 1, m)  
  Q_full <- c(0, Q)  
  area_into_piece <- target_area - Q_full[i]  
  rhs <- area_into_piece * a[i] + exp(a[i] * z[i] + b[i])  
  (log(rhs) - b[i]) / a[i] # invert piecewise-exp CDF  
}  
  
sample_fixed_random <- function(N, envelope) {  
  y <- sample_from_envelope_vec(envelope, N)  
  accept <- log(runif(N)) <= log_f_vec_faster(y) - eval_V(envelope, y)  
  y[accept]  
}  
sample_fixed_vec <- rng_vec(sample_fixed_random)
```

Finding the Best Number of Intervals

A fixed, non-adaptive grid of m tangent points, spaced over $[0, \hat{y} + 3\hat{\sigma}]$ (mode $\pm$ curvature-based spread). More points yields a higher acceptance rate, but each proposal also costs more to build and sample from.

```
time_for_m <- function(m, N, min_iterations = 5) {  
  y_points <- seq(0.02, y_hat + 3 * sd_hat, length.out = m)  
  envelope <- build_envelope(y_points)  
  b <- bench::mark(  
    sample_fixed_vec(N, envelope = envelope),  
    min_iterations = min_iterations, check = FALSE  
  )  
  data.frame(m = m, time = as.numeric(b$median), acceptance_rate = ...)  
}  
  
find_best_m <- function(candidate_ms, N, min_iterations = 5, acc_threshold = 0.  
  results <- do.call(rbind, lapply(candidate_ms, time_for_m, N = N,  
    min_iterations = min_iterations))  
  above <- results[results$acceptance_rate >= acc_threshold, ]  
  list(results = results, best = above[which.min(above$m), ])  
}  
  
out <- find_best_m(c(2, 3, 5, 8, 12, 20, 30, 50, 60, 70), N = 1000, min_iterati
```

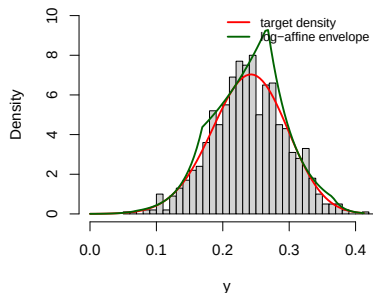
How the Best m Was Chosen



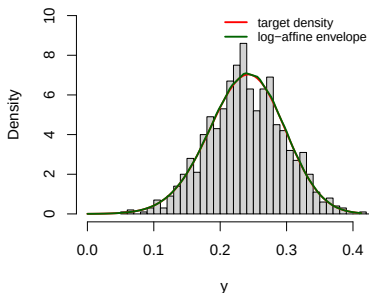
Time (left) drops sharply from $m = 2$ to $m = 5$, and is then mostly flat — re-running the sweep does not reliably pick the same argmin. Acceptance rate (right) is monotone and stable, so we take the smallest m crossing 99% (dashed line): $m = 20$ (red point).

Correctness Check: Log-Affine Envelope

m = 5: samples vs target vs envelope



m = 20: samples vs target vs envelope



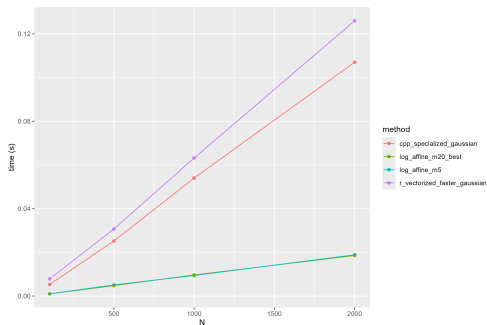
With $m = 5$ the envelope is visibly loose (lower acceptance rate); at the best m it hugs the target density much more tightly.

Benchmarking: Log-Affine vs. Gaussian



Comparing all four methods at $N = 100, 500, 1000, 2000$ (min. 20 iterations each) — both log-affine variants are consistently faster than either Gaussian implementation, and the gap widens as N grows.

Benchmarking Across N



Both log-affine variants beat `cpp_specialized_gaussian` and `r_vectorized_faster_gaussian` at every N, and the gap widens as N grows.

Benchmarking Across N: Why

Acceptance rate decides the speed — not the language.

Method	Acceptance
Gaussian envelope (R or C++)	$\approx 13\%$
Log-affine, $m = 5$	87.7%
Log-affine, best m	99.5%

Questions & Further Work

Questions?

Further work:

- ▶ The log-affine sampler is still written in R, while the Gaussian envelope also has a C++ version. Rewriting the log-affine sampler in C++ could make it even faster
- ▶ We placed the tangent points evenly spaced across the range. This may not be the best choice — spacing them based on where the curve is more curved could give a tighter envelope with fewer points